강화학습 4주차 과제

과목 : 강화학습

담당 교수 : 박태형 교수님

학과 : 지능로봇공학과

학번 : 2023042028

이름 : 권택주

목차

과제 정의

- Bellman Equation
- Dynamic Programming
  - Grid World
  - Policy & Value Iteration

과제 풀이

- Bellman Equation
- Dynamic Programming
  - Grid World
  - Policy & Value Iteration

과제 결과

과제 정의

Bellman optimality equation

- 현재 상태에서 얻을 수 있는 최대 기대 누적 보상
  - max를 사용해 가장 좋은 행동을 직접 선택
- $v_*(s) = \max_a \{r(s, a, s') + \gamma v_*(s')\}$

Policy & Value

- Policy가 고정되어 있을 때, 보상에 가까운 State부터 Value가 변화하고, 반복 계산을 통해 이전 State로 전파. 최종적으로 고정된 Policy를 따랐을 때 실제 기대 누적 보상은 $v_\pi(s)$에 수렴

과제 정의

Policy & Value Iteration

- Policy Iteration : Bellman equation으로 평가해 greedy로 policy를 개선하는 방식을 반복적으로 수행
- Value Iteration : Value를 계산하는 순간부터 가장 좋은 action 선택

과제 풀이

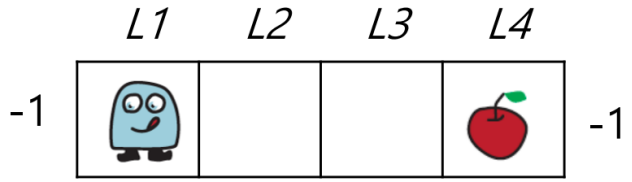
Bellman Equation Q1

(Q2) 앞 Example 의 2-grid World 문제를 확장한 4-Grid World 문제에 대하여, Bellman optimality equation 을 유도하라.

$\gamma = 0.9$

$V_*(s) = \max_a \{ r(s, a, s') + \gamma V_*(s') \}$

s	s'	a	r
L_1	L_1	left	-1
L_1	L_2	right	0
L_2	L_1	left	0
L_2	L_3	right	0
L_3	L_2	left	0
L_3	L_4	right	1
L_4	L_3	left	0
L_4	L_4	right	-1



$V_*(L_1) = \max(-1 + 0.9V_*(L_1), 0.9V_*(L_2)) \Rightarrow 0.9V(L_2)$.

$V_*(L_2) = \max(0.9V_*(L_1), 0.9V_*(L_3)) \Rightarrow 0.9V(L_1)$ → 보상 L_4에 더 가까워짐.

$V_*(L_3) = \max(0.9V_*(L_2), 1 + 0.9V_*(L_4)) \Rightarrow 1 + 0.9V(L_4)$

$V(L_4) = \max(0.9V_*(L_3), -1 + 0.9V_*(L_4)) \Rightarrow 0.9V(L_3)$

$V(L_3) = 1 + 0.81V(L_3)$, $0.19V(L_3) = 1$, $V_*(L_3) = 5.263$.

$V_*(L_4) = 4.7369$

$V_*(L_2) = 0.9V(L_3)$
$= 4.7367$.

$V_*(L_1) = 0.9V(L_2)$
$= 0.9 \times 4.7367$
$= 4.263$

Optimal State Value
- $V_*(L_1) = 4.263$
- $V_*(L_2) = 4.737$
- $V_*(L_3) = 5.263$
- $V_*(L_4) = 4.737$

Optimal Policy

L_1	L_2	L_3	L_4
→	→	→	←

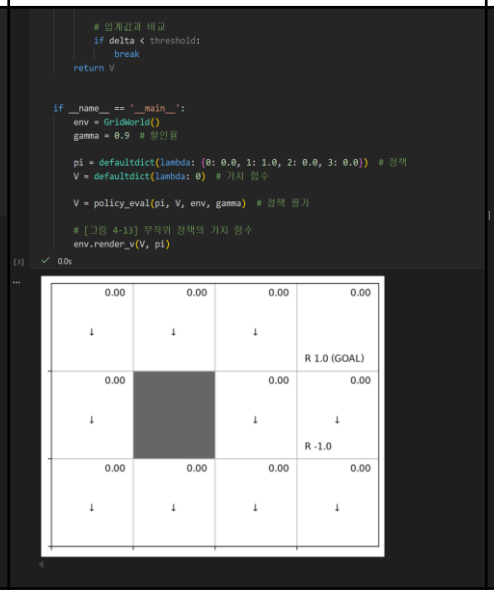
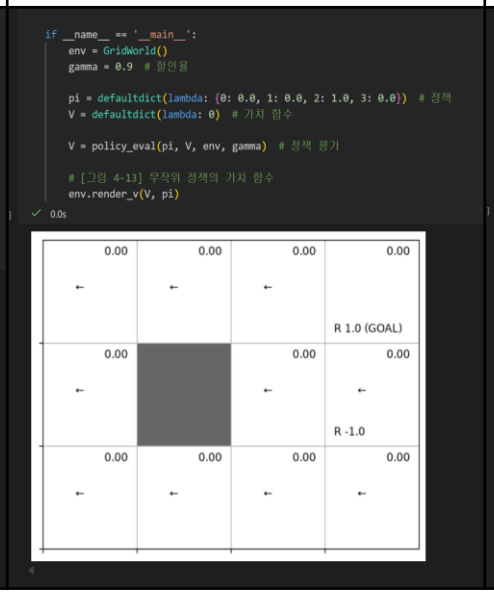
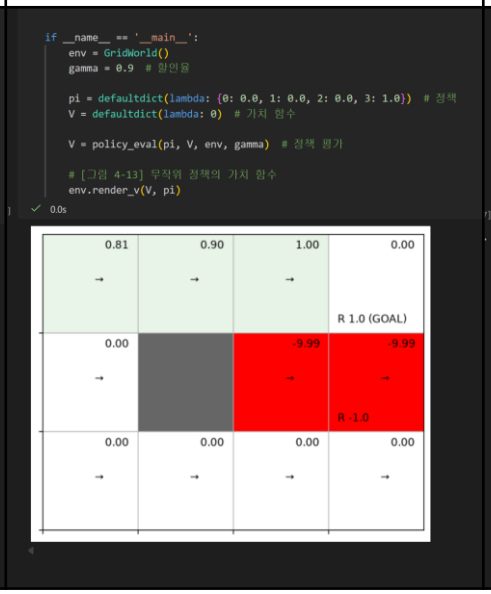
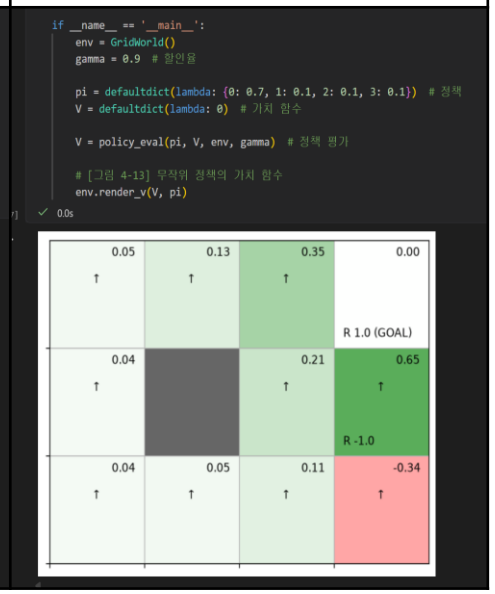
$v_x(L_2)\max(0.9v_*(L1), 0.9v_*(L2))$에서 $0.9v_*(L2)$를 선택한 이유는 보상 L_4에 더 가깝게 위치해 있어 max 값으로 판단 되기 때문이다.

Optimal State Value
- $v_*(L_1) = 4.263$
- $v_*(L_2) = 4.739$
- $v_*(L_3) = 5.263$
- $v_*(L_4) = 4.737$

과제 풀이

Dynamic Programming Q1

- Policy 변화에 따른 Value값 분석

[1, 0, 0, 0]	[0, 1, 0, 0]	[0, 0, 1, 0]	[0, 0, 0, 1]	[0.7, 0.1, 0.1, 0.1]
				

과제 풀이

Dynamic Programming Q1

- Policy 변화에 따른 Value 분석
- 1번부터 4번까지 문제는 항상 특정 방향으로만 움직여 보상을 받는 구조
  - 1번 : Goal 방향 상태의 V 증가, -1에서 빠져나오지 못하는 상태는 약 -10
  - 2번 : Goal/패널티에 접근하지 않아 0
  - 3번 : Goal/패널티에 접근하지 않아 0
  - 4번 : Goal 바로 아래는 1. -1 아래는 -1 + 0.9*1 = -0.1
- 5번 문제는 특정 방향을 보는 것이 아닌, 4 방향 중 위쪽 행동에 강하게 편향된 확률 정책
  - 여러 경로의 기대값이 혼합되어 Value가 전체 Grid에 전파

과제 풀이

Dynamic Programming Q2 – Policy

- **Policy Iteratioin**
  - 현재 policy 평가 -> 해당 Policy에서 V 수렴 -> V 기준으로 Greedy Policy 생성
  - 5번째 반복에서 이전 정책과 새로 계산된 정책이 동일해 수렴
  - 각 반복 안에서 Value를 평가해 비교적 적은 외부 반복 횟수로 최적 정책에 도달

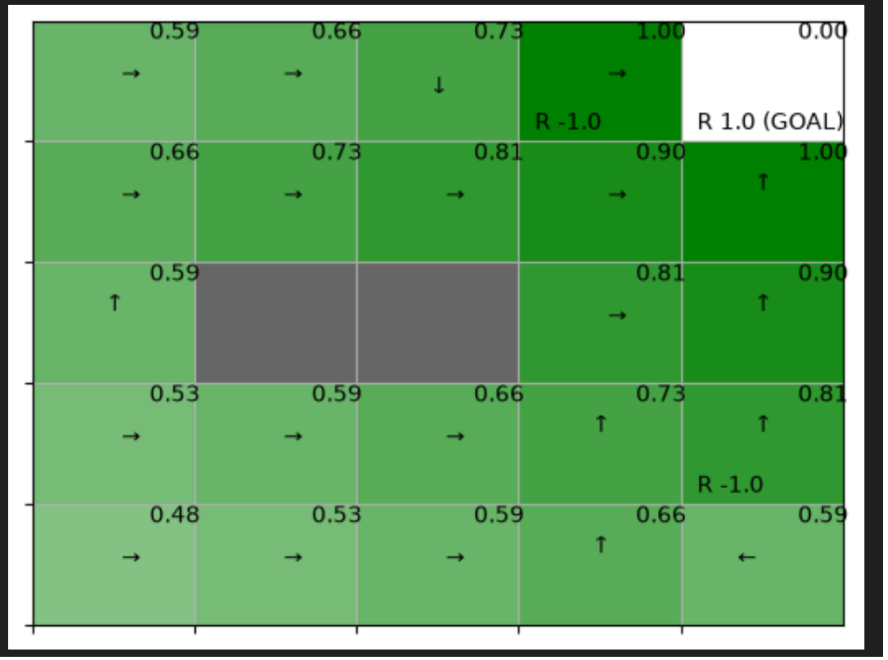
```
if __name__ == '__main__':

    env = GridWorld()
    gamma = 0.9
    pi, V = policy_iter(env, gamma)

    # 최종 결과 출력
    env.render_v(V, pi)
```

[2]

```
Iteration 0: policy updated
Iteration 1: policy updated
Iteration 2: policy updated
Iteration 3: policy updated
Iteration 4: policy updated
Policy converged after 5 iterations.
```



과제 풀이

Dynamic Programming Q2 – Value

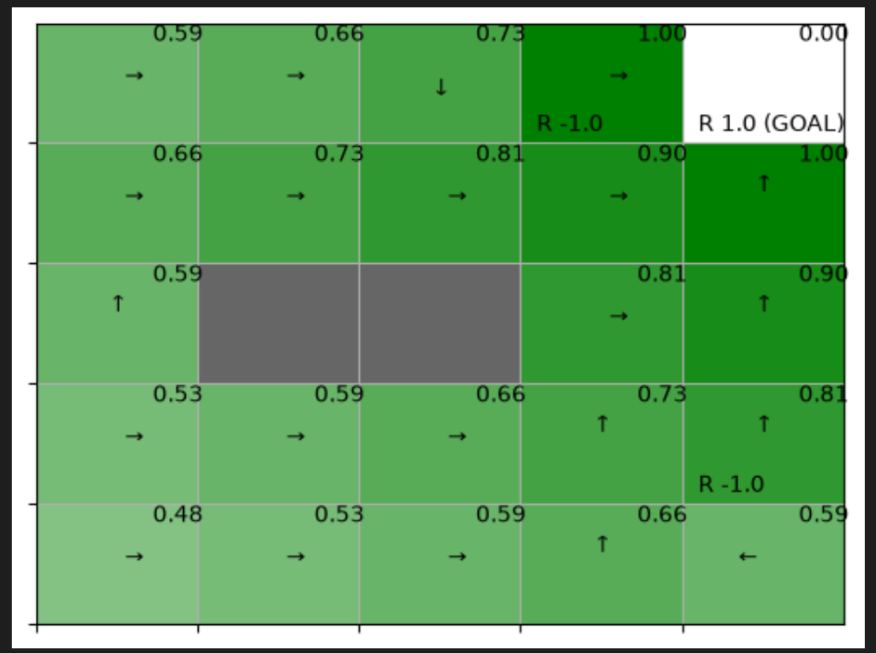
- **Value Iteration**
  - 모든 State를 한번 순회 -> 각 State의 V를 한 단계 갱신
  - 초기 값이 V(s) = 0. Goal의 보상이 주변 상태에 영향
  - 이후 반복하면서 해당 정보가 먼 상태까지 전달
  - 모든 Value의 최대 변화량이 0.001보다 작아질 때 까지 반복
  - 7번째 반복에서 최대 변화량 Delta가 설정된 threshold 0.001보다 작아져 수렴

```
if __name__ == '__main__':
    V = defaultdict(lambda: 0)
    env = GridWorld()
    gamma = 0.9

    V = value_iter(V, env, gamma)  # 최적 가치 함수 찾기
    pi = greedy_policy(V, env, gamma)  # 최적 정책 찾기
    env.render_v(V, pi)
```

[3]

```
Iteration 1: value updated, delta = 1.000000
Iteration 2: value updated, delta = 0.900000
Iteration 3: value updated, delta = 0.810000
Iteration 4: value updated, delta = 0.729000
Iteration 5: value updated, delta = 0.656100
Iteration 6: value updated, delta = 0.590490
Value converged after 7 iterations.
```



과제 결과

1. **Bellman optimality equation**
   - Action별 값을 비교해 max를 선택하고 Optimal State Value 및 Policy 도출
2. **Policy 변화에 따른 Value**
   - 동일 환경에서 Policy에 따라 State Valeu 값 변화
   - 확률적 Policy에서는 행동 확률에 따른 기대 보상이 Value 에 반영
3. **Policy Iteration**
   - Policy Evaluation과 Greedy Policy Improvement를 반복
4. **Value Iteration**
   - State Value를 반복적으로 직접 갱신